

Cruncherx (Root)

- A. .streamlit
- B. .supabase-venv
- C. app

Home.py, requirements.txt, local-requirements.txt, README.md, bulldozer.txt

A. .streamlit

1. Config.toml

```
[theme]
primaryColor="#39FF14"
backgroundColor="#000000"
secondaryBackgroundColor="#111111"
textColor="#FFFFFF"
font="sans serif"
```

2. runtime.txt

3.11

3. secrets.toml

```
SUPABASE_URL = "https://uphjhkhdpqbntmxfysjg.supabase.co"
SUPABASE_KEY = "
"
"
"
"
```

4.

B. .supabase-venv

- 1. Include
- 2. Lib
- 3. Scripts
- 4. pyvenv.cfg

```
home = C:\ProgramData\anaconda3
include-system-site-packages = false
version = 3.12.7
```

```
executable = C:\ProgramData\anaconda3\python.exe
```

```
command = C:\ProgramData\anaconda3\python.exe -m venv C:\Users\jagdevsinghdosanjh.RHINO\CruncherX\supabase-venv
```

C. app

- a. assets
- b. backend
 - 1. api
 - 2. auth
 - 3. backup
 - 4. billing
 - 5. monitoring
 - 6. orgs
 - 7. self_heal.py

```
import os
import re
```

```
# Pattern that matches Edge sidebar injected metadata
EDGE_METADATA_PATTERN = re.compile(
    r"# User's Edge browser tabs
    metadata[\\s\\S]*?edge_all_open_tabs\\s*=\\s*[\\[.]*?\\]",
    re.MULTILINE
)
```

```
def clean_file(path: str):
    """Remove Edge metadata from a single file."""
    try:
        with open(path, "r", encoding="utf-8") as f:
            content = f.read()

        # If no metadata found, skip
        if "edge_all_open_tabs" not in content:
            return False

        # Remove metadata block
        cleaned = re.sub(EDGE_METADATA_PATTERN, "", content)

        # Write cleaned file
        with open(path, "w", encoding="utf-8") as f:
            f.write(cleaned)
```

```

        print(f"[SELF-HEAL] Cleaned metadata from: {path}")
        return True

    except Exception as e:
        print(f"[SELF-HEAL ERROR] Could not clean {path}: {e}")
        return False

def heal_project(root="app"):
    """Scan entire CruncherX project and remove Edge metadata."""
    healed_files = 0

    for subdir, _, files in os.walk(root):
        for file in files:
            if file.endswith(".py"):
                full_path = os.path.join(subdir, file)
                if clean_file(full_path):
                    healed_files += 1

    print(f"[SELF-HEAL] Completed. Files healed: {healed_files}")
    return healed_files

```

8. subscriptions.py

```

from typing import Optional, Dict, Any
import streamlit as st
from backend.supabase_client import get_supabase_client
from datetime import datetime, date, timezone

# -----
# Supabase Client (cached)
# -----
sb = get_supabase_client()

# -----
# 1. Get User + Plan (matches your actual schema)
# -----
def get_user_plan(user_id: str) -> Optional[Dict[str, Any]]:
    """
    Loads the user's active subscription plan based on:

```

```
- profiles.plan_id
- subscription_plans table
"""
```

```
# Fetch profile
```

```
profile_res = (
    sb.table("profiles")
    .select("*")
    .eq("id", user_id)
    .maybe_single()
    .execute()
)
```

```
if profile_res is None:
    return None
```

```
profile = getattr(profile_res, "data", profile_res)
if not isinstance(profile, dict):
    return None
```

```
plan_id = profile.get("plan_id")
if not plan_id:
    return None
```

```
# Fetch plan details
```

```
plan_res = (
    sb.table("subscription_plans")
    .select("*")
    .eq("id", plan_id)
    .maybe_single()
    .execute()
)
```

```
if plan_res is None:
    return None
```

```
plan = getattr(plan_res, "data", plan_res)
if not isinstance(plan, dict):
    return None
```

```

return {
    "id": profile.get("id"),
    "email": profile.get("email"),
    "plan_id": plan_id,
    "plan_expiry": profile.get("plan_expiry"),
    "plan": plan,
}

# -----
# 2. Check if plan expired
# -----
def is_plan_expired(user):
    expiry = user.get("plan_expiry")
    if not expiry:
        return True

    # Convert Supabase timestamp to aware datetime
    expiry_dt = datetime.fromisoformat(expiry)

    # Ensure UTC timezone awareness
    if expiry_dt.tzinfo is None:
        expiry_dt = expiry_dt.replace(tzinfo=datetime.timezone.utc)

    # Compare with aware UTC now
    now_utc = datetime.now(datetime.timezone.utc)

    return expiry_dt < now_utc

# def is_plan_expired(user: Dict[str, Any]) -> bool:
#     expiry = user.get("plan_expiry")
#     if not expiry:
#         return True

#     try:
#         expiry_dt = datetime.fromisoformat(expiry.replace("Z", ""))
#     except Exception:
#         return True

#     return expiry_dt < datetime.utcnow()

```

```

# -----
# 3. Daily Limit Check (usage_logs table)
# -----
def check_daily_limit(user_id: str) -> int:
    today_str = date.today().isoformat()

    res = (
        sb.table("usage_logs")
        .select("id")
        .eq("user_id", user_id)
        .eq("action", "compress")
        .gte("created_at", today_str)
        .execute()
    )

    data = getattr(res, "data", res)
    if not isinstance(data, list):
        return 0

    return len(data)

# -----
# 4. Enforce Plan Rules (Free / Basic / Pro / Business)
# -----
def enforce_plan_rules(user: Dict[str, Any]) -> Dict[str, Any]:
    plan_name = user["plan"].get("name")

    if not plan_name:
        return {"allowed": False, "reason": "Invalid plan."}

    if is_plan_expired(user):
        return {"allowed": False, "reason": "Plan expired. Please renew."}

    # ----- FREE PLAN -----
    if plan_name == "Free":
        if check_daily_limit(user["id"]) >= 5:
            return {"allowed": False, "reason": "Daily limit reached (5/day)."}
        return {
            "allowed": True,
            "watermark": True,

```

```

        "priority": False,
        "bulk": False,
        "api": False,
    }

# ----- BASIC PLAN -----
if plan_name == "Basic":
    return {
        "allowed": True,
        "watermark": False,
        "priority": False,
        "bulk": True,
        "api": False,
    }

# ----- PRO PLAN -----
if plan_name == "Pro":
    return {
        "allowed": True,
        "watermark": False,
        "priority": True,
        "bulk": True,
        "api": False,
    }

# ----- BUSINESS PLAN -----
if plan_name == "Business":
    return {
        "allowed": True,
        "watermark": False,
        "priority": True,
        "bulk": True,
        "api": True,
    }

return {"allowed": False, "reason": "Unknown plan."}

# -----
# 5. Log Usage (matches usage_logs table)
# -----

```

```
def log_usage(user_id: str, org_id: str, action: str, bytes_in: int, bytes_out:
int):
    sb.table("usage_logs").insert({
        "user_id": user_id,
        "org_id": org_id,
        "action": action,
        "bytes_in": bytes_in,
        "bytes_out": bytes_out,
    }).execute()
```

9. supabase_client.py

```
import streamlit as st
from supabase import create_client

@st.cache_resource
def get_supabase_client():
    url = st.secrets["SUPABASE_URL"]
    key = st.secrets["SUPABASE_KEY"] # ✓ correct key name
    return create_client(url, key)
```

10. usage_logger.py

```
import streamlit as st
from supabase import create_client

@st.cache_resource
def supabase():
    return create_client(st.secrets["SUPABASE_URL"],
st.secrets["SUPABASE_KEY"])

def log_engine_run(user_id, org_id, engine_type, input_bytes,
output_bytes, status):
    sb = supabase()

    compression_ratio = None
    if input_bytes and output_bytes:
        compression_ratio = output_bytes / input_bytes
```



```

sb.table("engine_logs").insert({
    "user_id": user_id,
    "org_id": org_id,
    "engine_type": engine_type,
    "input_bytes": input_bytes,
    "output_bytes": output_bytes,
    "compression_ratio": compression_ratio,
    "status": status
}).execute()

```

c. components

1. footer.py

```

import streamlit as st

def render_footer():
    st.markdown(
        """
        <div style='
            text-align:center;
            color:#39FF14;
            font-size:13px;
            padding:12px 0;
            background-color:#000000;
            border-top:1px solid #39FF14;
        '>
            Made in Punjab IN by <b>Jagdev Singh Dosanjh</b> — GHS
            Chananke<br>
            <span style='opacity:0.85;'>CruncherX — Black & Neon Green
            Edition</span>
        </div>
        """,
        unsafe_allow_html=True
    )

```

2. sidebar.py

```

import streamlit as st

```

```

def render_sidebar():
    st.sidebar.markdown("""
    <div style='text-align:center; padding:15px'>
        <h1 style='color:#39FF14; margin-bottom:0;'>CruncherX</h1>
        <p style='color:#AAAAAA; margin-top:4px;'>PDF Compression
Suite</p>
        <hr style='border-color:#39FF14;'>
    </div>
    """, unsafe_allow_html=True)

```

d. engines

1. compression_engine.py

```

import time

def compress_pdf(input_bytes, quality, supabase, user_id, org_id):
    start = time.time()

    try:
        # --- your actual compression logic here ---
        compressed_bytes = actual_compression_logic(input_bytes, quality)

        status = "success"
        output_bytes = len(compressed_bytes)

    except Exception as e:
        status = "error"
        output_bytes = None

        # Log error
        supabase.table("error_logs").insert({
            "error_message": str(e),
            "error_type": type(e).__name__
        }).execute()

        compressed_bytes = None

    end = time.time()

```

```

duration_ms = (end - start) * 1000

# Log latency
supabase.table("latency_stats").insert({
    "engine_type": "cloud",
    "duration_ms": duration_ms,
    "stage": "compress"
}).execute()

# Log engine run
supabase.table("engine_logs").insert({
    "user_id": user_id,
    "org_id": org_id,
    "engine_type": "cloud",
    "input_bytes": len(input_bytes),
    "output_bytes": output_bytes,
    "compression_ratio": (output_bytes / len(input_bytes)) if
output_bytes else None,
    "status": status
}).execute()

return compressed_bytes

```

2. cruncher_clous.py

```

import fitz
from PIL import Image
import io
import os
import time
from backend.usage_logger import log_engine_run

def compress_to_target(input_path, supabase, user_id, org_id,
target_mb=7):
    start = time.time()

    try:
        base = os.path.splitext(os.path.basename(input_path))[0]
        output_path = f"{base}_cloud.pdf"

```

```

doc = fitz.open(input_path)
new_pdf = fitz.open()

for page in doc:
    pix = page.get_pixmap(dpi=72)
    img = Image.frombytes("RGB", (pix.width, pix.height), pix.samples)

    buf = io.BytesIO()
    img.save(buf, format="JPEG", quality=50, optimize=True)
    buf.seek(0)

    img_pdf = fitz.open()
    rect = fitz.Rect(0, 0, pix.width, pix.height)
    p = img_pdf.new_page(width=pix.width, height=pix.height)
    p.insert_image(rect, stream=buf.read())
    new_pdf.insert_pdf(img_pdf)

new_pdf.save(output_path, deflate=True, garbage=3)
new_pdf.close()

output_bytes = os.path.getsize(output_path)
status = "success"

except Exception as e:
    status = "error"
    output_bytes = None

    supabase.table("error_logs").insert({
        "error_message": str(e),
        "error_type": type(e).__name__
    }).execute()

    return None, None, "CloudError"

end = time.time()
duration_ms = (end - start) * 1000

# Log latency
supabase.table("latency_stats").insert({
    "engine_type": "cloud",

```

```

        "duration_ms": duration_ms,
        "stage": "compress"
    }).execute()

# Log engine run
log_engine_run(
    user_id=user_id,
    org_id= org_id if org_id else None,
    engine_type="cloud",
    input_bytes=os.path.getsize(input_path),
    output_bytes=output_bytes,
    status=status
)

size_mb = output_bytes / (1024 * 1024)
return output_path, size_mb, "CloudSafe"

```

3. cruncher_local.py

```

import fitz
from PIL import Image
import io
import os
import time

def compress_to_target_local(input_path, target_mb=7):
    start = time.time()

    try:
        base = os.path.splitext(os.path.basename(input_path))[0]
        output_path = f"{base}_local.pdf"

        doc = fitz.open(input_path)
        new_pdf = fitz.open()

        for page in doc:
            pix = page.get_pixmap(dpi=72)
            img = Image.frombytes("RGB", (pix.width, pix.height), pix.samples)

```

```

buf = io.BytesIO()
img.save(buf, format="JPEG", quality=50, optimize=True)
buf.seek(0)

img_pdf = fitz.open()
rect = fitz.Rect(0, 0, pix.width, pix.height)
p = img_pdf.new_page(width=pix.width, height=pix.height)
p.insert_image(rect, stream=buf.read())
new_pdf.insert_pdf(img_pdf)

new_pdf.save(output_path, deflate=True, garbage=3)
new_pdf.close()

output_bytes = os.path.getsize(output_path)
status = "success"

except Exception as e:
    print("Local Engine Error:", e)
    return None, None, "LocalError"

end = time.time()
duration_ms = (end - start) * 1000

print(f"Local compression completed in {duration_ms:.2f} ms")

size_mb = output_bytes / (1024 * 1024)
return output_path, size_mb, "LocalSafe"

```

e. pages

1. Admin_Dashboard.py

```

import streamlit as st
from backend.monitoring.stats import get_basic_stats
from backend.supabase_client import get_supabase_client

st.set_page_config(page_title="Admin Dashboard", layout="wide")

# -----

```

```

# ADMIN ACCESS CONTROL
# -----
ADMIN_EMAILS = ["jagdevsinghdosanjh@gmail.com"]

def require_admin():
    if "current_user" not in st.session_state:
        st.error("You must be logged in.")
        st.stop()

    user = st.session_state["current_user"]

    if user["email"] not in ADMIN_EMAILS:
        st.error("Access denied. Admins only.")
        st.stop()

# -----
# MAIN DASHBOARD
# -----
def main():
    require_admin()

    st.title("🔒 Admin Dashboard")

    supabase = get_supabase_client()

    # FIXED: get_basic_stats() takes NO arguments
    stats = get_basic_stats()

    st.markdown("### 🔍 System Overview")

    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Total Jobs", stats.get("total_jobs", 0))
    c2.metric("Cloud Jobs", stats.get("cloud_jobs", 0))
    c3.metric("Local Jobs", stats.get("local_jobs", 0))
    c4.metric("Total Errors", stats.get("total_errors", 0))

    st.markdown("---")
    st.markdown("### 📦 Raw Tables")

    col_left, col_right = st.columns(2)

```

```

# ----- ENGINE LOGS -----
with col_left:
    st.subheader("Engine Logs")
    try:
        logs = (
            supabase.table("engine_logs")
            .select("*")
            .order("created_at", desc=True)
            .limit(50)
            .execute()
        )
        st.dataframe(logs.data or [], height=300)
    except Exception as e:
        st.error(f"Error loading engine_logs: {e}")

# ----- ERROR LOGS -----
with col_right:
    st.subheader("Error Logs")
    try:
        errors = (
            supabase.table("error_logs")
            .select("*")
            .order("created_at", desc=True)
            .limit(50)
            .execute()
        )
        st.dataframe(errors.data or [], height=300)
    except Exception as e:
        st.error(f"Error loading error_logs: {e}")

if __name__ == "__main__":
    main()

```

2. Cloud_Compressor.py

```

import streamlit as st
import uuid
import os
import tempfile

```



```

import gc
import time

from components.sidebar import render_sidebar
from components.footer import render_footer

from engines.cruncher_cloud import compress_to_target
from backend.subscriptions import (
    get_user_plan,
    enforce_plan_rules,
    log_usage,
)
from backend.supabase_client import get_supabase_client
from auth import get_current_user

# ----- UI -----
render_sidebar()
st.title("☁ Cloud Compressor")

# ----- AUTH -----
user = get_current_user()
if user is None:
    st.switch_page("Home.py")

USER_ID = user["id"]
ORG_ID=None
#ORG_ID = "" # Reserved for future org support

sb = get_supabase_client()

# ----- PLAN LOADING -----
plan = get_user_plan(USER_ID)

if plan is None:
    st.error("No active plan found.")
    st.stop()

st.info(f"Your Plan: **{plan['plan']['name']}**")

# ----- FILE UPLOAD -----

```

```

uploaded_files = st.file_uploader(
    "Upload PDFs",
    type=["pdf"],
    accept_multiple_files=True
)

# ----- PROCESSING -----
if uploaded_files:
    if st.button("Start Cloud Compression"):

        for file in uploaded_files:
            st.write(f"📄 Processing: **{file.name}**")

            # Enforce plan rules
            rules = enforce_plan_rules(plan)
            if not rules.get("allowed", False):
                st.error(rules.get("reason"))
                continue

            # Save uploaded file to temp
            temp_dir = tempfile.gettempdir()
            unique_name = f"{uuid.uuid4()}_{file.name}"
            input_path = os.path.join(temp_dir, unique_name)

            with open(input_path, "wb") as f:
                f.write(file.getvalue())

            # Release Streamlit's file handle and force GC (Windows lock fix)
            file = None
            gc.collect()

            # Run cloud engine
            with st.spinner("Compressing in cloud..."):
                try:
                    output_path, size_mb, mode = compress_to_target(
                        input_path,
                        sb,
                        USER_ID,
                        ORG_ID,
                    )

```

```

except Exception as e:
    st.error(f"Cloud compression failed: {e}")
    # Best-effort cleanup
    if os.path.exists(input_path):
        try:
            os.remove(input_path)
        except PermissionError:
            time.sleep(0.2)
        try:
            os.remove(input_path)
        except:
            pass
    continue

if output_path is None:
    st.error("Cloud engine returned no output.")
    # Cleanup input file
    if os.path.exists(input_path):
        try:
            os.remove(input_path)
        except PermissionError:
            time.sleep(0.2)
        try:
            os.remove(input_path)
        except:
            pass
    continue

# Show result
st.success(f"Compressed Size: {size_mb:.2f} MB ({mode})")

# Download button
with open(output_path, "rb") as f_out:
    st.download_button(
        label=f"↓ Download {file.name if file else 'compressed.pdf'}",
        data=f_out.read(),
        file_name=f"crunched_{unique_name.split('_', 1)[-1]}",
        mime="application/pdf"
    )

```

```

# Log usage
try:
    log_usage(
        user_id=USER_ID,
        org_id=ORG_ID,
        action="compress",
        bytes_in=os.path.getsize(input_path),
        bytes_out=os.path.getsize(output_path),
    )
except Exception as e:
    st.warning(f"Usage logging failed: {e}")

# Cleanup input/output files with Windows-safe handling
if os.path.exists(input_path):
    try:
        os.remove(input_path)
    except PermissionError:
        time.sleep(0.2)
    try:
        os.remove(input_path)
    except:
        pass

if os.path.exists(output_path):
    try:
        os.remove(output_path)
    except:
        pass

# ----- FOOTER -----
render_footer()

```

3. Local_Compressor.py

```

import streamlit as st
import uuid
import os
import tempfile
import gc

```

```

import time

from components.sidebar import render_sidebar
from components.footer import render_footer

from engines.cruncher_local import compress_to_target_local

# ----- UI -----
render_sidebar()
st.title(" 📁 Local Compressor")

# ----- FILE UPLOAD -----
uploaded_files = st.file_uploader(
    "Upload PDFs",
    type=["pdf"],
    accept_multiple_files=True
)

# ----- PROCESSING -----
if uploaded_files:
    if st.button("Start Local Compression"):

        for file in uploaded_files:
            st.write(f" 📄 Processing: **{file.name}**")

            # Save uploaded file to temp
            temp_dir = tempfile.gettempdir()
            unique_name = f"{uuid.uuid4()}_{file.name}"
            input_path = os.path.join(temp_dir, unique_name)

            with open(input_path, "wb") as f:
                f.write(file.getvalue())

            # Release Streamlit's file handle and force GC (Windows lock fix)
            file = None
            gc.collect()

            # Run local engine
            with st.spinner("Compressing locally..."):
                try:

```

```

        output_path, size_mb, mode = compress_to_target_local(
            input_path,
            target_mb=7
        )
except Exception as e:
    st.error(f"Local compression failed: {e}")
    # Best-effort cleanup
    if os.path.exists(input_path):
        try:
            os.remove(input_path)
        except PermissionError:
            time.sleep(0.2)
        try:
            os.remove(input_path)
        except:
            pass
    continue

if output_path is None:
    st.error("Local engine returned no output.")
    # Cleanup input file
    if os.path.exists(input_path):
        try:
            os.remove(input_path)
        except PermissionError:
            time.sleep(0.2)
        try:
            os.remove(input_path)
        except:
            pass
    continue

# Show result
st.success(f"Compressed Size: {size_mb:.2f} MB ({mode})")

# Download button
with open(output_path, "rb") as f_out:
    st.download_button(
        label=f"↓ Download {unique_name.split('_', 1)[-1]}",
        data=f_out.read(),

```

```

        file_name=f"local_{unique_name.split('_', 1)[-1]}",
        mime="application/pdf"
    )

    # Cleanup input/output files with Windows-safe handling
    if os.path.exists(input_path):
        try:
            os.remove(input_path)
        except PermissionError:
            time.sleep(0.2)
        try:
            os.remove(input_path)
        except:
            pass

    if os.path.exists(output_path):
        try:
            os.remove(output_path)
        except:
            pass

    # ----- FOOTER -----
    render_footer()

```

4. Organization_Management.py

No Code here as yet

5. API_Keys.py

No code here as yet

6. OCR_Coming_Soon.py

```

import streamlit as st
from components.sidebar import render_sidebar
from components.footer import render_footer
from pdf2image import convert_from_path
import pytesseract
import fitz
import os
import tempfile

```

```

st.set_page_config(page_title="OCR Engine", layout="wide")

render_sidebar()
st.title("🔍 OCR Engine — Searchable PDF Generator")

st.write("Convert scanned PDFs into **searchable, OCR-enabled PDFs**  
using Tesseract OCR.")

uploaded_file = st.file_uploader("Upload scanned PDF", type=["pdf"])

if uploaded_file:
    with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as tmp:
        tmp.write(uploaded_file.read())
        input_path = tmp.name

    st.info("Extracting pages...")
    images = convert_from_path(input_path, dpi=300)

    st.info("Running OCR on pages...")
    ocr_pdf = fitz.open()

    for idx, img in enumerate(images):
        st.write(f"OCR processing page {idx+1}/{len(images)}...")

        # Convert PIL image → bytes
        img_bytes = img.tobytes()

        # OCR text
        text = pytesseract.image_to_string(img)

        # Create PDF page
        rect = fitz.Rect(0, 0, img.width, img.height)
        page = ocr_pdf.new_page(width=img.width, height=img.height)

        # Insert original image
        page.insert_image(rect, pixmap=fitz.Pixmap(fitz.csRGB, img.width,
img.height, img_bytes))

        # Insert invisible OCR text layer

```



```

        page.insert_textbox(rect, text, fontsize=12, overlay=False)

    output_path = "ocr_output.pdf"
    ocr_pdf.save(output_path)
    ocr_pdf.close()

    st.success("OCR complete — your searchable PDF is ready!")

    with open(output_path, "rb") as f:
        st.download_button(
            label="Download Searchable PDF",
            data=f.read(),
            file_name="ocr_output.pdf",
            mime="application/pdf"
        )

    render_footer()

```

Note: I think file name change is required here now.

7. About_CruncherX.py

```

import streamlit as st
from components.sidebar import render_sidebar
from components.footer import render_footer

render_sidebar()

st.title("📄 About CruncherX")

st.markdown("""
**CruncherX** is a dual-engine PDF compression suite designed for
speed, clarity, and reliability.

### 🚀 What CruncherX Offers
- ☁️ **Cloud Engine** — Safe, fast online compression
- 💻 **Local Bulldozer Engine** — Maximum offline compression
power
- 📊 **Smart Size Targeting** — Automatically compresses PDFs under 7
MB

```

- 📄 ****Perfect for Scanned PDFs**** — Ideal for BLOs, schools, offices, and admins

CruncherX is built for professionals who handle large scanned PDFs daily and need consistent, high-quality compression.

```
""")
```

```
render_footer()
```

8. Monitoring.py

```
import streamlit as st
import pandas as pd
import altair as alt
from backend.supabase_client import get_supabase_client

st.set_page_config(page_title="Monitoring Dashboard", layout="wide")

def load_table(supabase, table):
    try:
        data = supabase.table(table).select("*").order("created_at",
desc=True).execute()
        return pd.DataFrame(data.data) if data.data else pd.DataFrame()
    except Exception:
        return pd.DataFrame()

def safe_col(df, col):
    return col in df.columns and not df.empty

def main():
    st.title("📊 CruncherX Monitoring Dashboard")

    supabase = get_supabase_client()

    # Load all monitoring tables
    engine_logs = load_table(supabase, "engine_logs")
    latency_stats = load_table(supabase, "latency_stats")
    error_logs = load_table(supabase, "error_logs")
    job_metrics = load_table(supabase, "job_metrics")
```

```

st.markdown("### 🔍 System Overview")

col1, col2, col3, col4 = st.columns(4)

# ===== SAFE METRICS =====
total_jobs = len(engine_logs)

cloud_jobs = (
    len(engine_logs[engine_logs["engine_type"] == "cloud"])
    if safe_col(engine_logs, "engine_type")
    else 0
)

local_jobs = (
    len(engine_logs[engine_logs["engine_type"] == "local"])
    if safe_col(engine_logs, "engine_type")
    else 0
)

total_errors = len(error_logs)

col1.metric("Total Jobs", total_jobs)
col2.metric("Cloud Jobs", cloud_jobs)
col3.metric("Local Jobs", local_jobs)
col4.metric("Errors", total_errors)

st.markdown("---")

# =====
# ENGINE PERFORMANCE CHART
# =====

st.subheader("⚙️ Engine Performance (Compression Ratio Over Time)")

if safe_col(engine_logs, "compression_ratio") and safe_col(engine_logs,
"created_at"):
    engine_logs["created_at"] =
pd.to_datetime(engine_logs["created_at"])

```

```

chart = (
    alt.Chart(engine_logs)
    .mark_line(point=True)
    .encode(
        x="created_at:T",
        y="compression_ratio:Q",
        color="engine_type:N",
        tooltip=[
            "engine_type",
            "input_bytes",
            "output_bytes",
            "compression_ratio",
            "created_at",
        ],
    )
    .properties(height=300)
)

```

```

st.altair_chart(chart, width="stretch")
else:
    st.info("No engine logs yet.")

```

```

st.markdown("----")

```

```

# =====
# LATENCY CHART
# =====
st.subheader("🕒 Latency Over Time")

```

```

if safe_col(latency_stats, "duration_ms") and safe_col(latency_stats,
"created_at"):
    latency_stats["created_at"] =
pd.to_datetime(latency_stats["created_at"])

```

```

chart = (
    alt.Chart(latency_stats)
    .mark_area(opacity=0.4)
    .encode(
        x="created_at:T",
        y="duration_ms:Q",
    )

```

```

        color="engine_type:N",
        tooltip=["engine_type", "duration_ms", "stage", "created_at"],
    )
    .properties(height=300)
)

st.altair_chart(chart, width="stretch")
else:
    st.info("No latency data yet.")

st.markdown("----")

# =====
# ERROR LOGS TABLE
# =====
st.subheader(" ❌ Error Logs")

if not error_logs.empty:
    st.dataframe(error_logs, width="stretch", height=250)
else:
    st.success("No errors recorded.")

st.markdown("----")

# =====
# JOB METRICS (CPU + RAM)
# =====
st.subheader(" 📊 System Resource Usage (Local Engine)")

if safe_col(job_metrics, "cpu_usage") and safe_col(job_metrics,
"created_at"):
    job_metrics["created_at"] =
pd.to_datetime(job_metrics["created_at"])

cpu_chart = (
    alt.Chart(job_metrics)
    .mark_line(point=True)
    .encode(
        x="created_at:T",
        y="cpu_usage:Q",

```

```

        tooltip=["cpu_usage", "memory_usage", "created_at"],
    )
    .properties(title="CPU Usage (%)", height=250)
)

```

```

mem_chart = (
    alt.Chart(job_metrics)
    .mark_line(point=True, color="orange")
    .encode(
        x="created_at:T",
        y="memory_usage:Q",
        tooltip=["cpu_usage", "memory_usage", "created_at"],
    )
    .properties(title="Memory Usage (MB)", height=250)
)

```

```

st.altair_chart(cpu_chart, width="stretch")
st.altair_chart(mem_chart, width="stretch")

```

```

else:
    st.info("No job metrics yet.")

```

```

st.markdown("----")

```

```

st.caption("CruncherX Monitoring System — Powered by Supabase + Streamlit")

```

```

if __name__ == "__main__":
    main()

```

9. Billing.py

No code here as yet

10. Contact_Support.py

```

import streamlit as st
from components.sidebar import render_sidebar
from components.footer import render_footer

```

```

render_sidebar()
st.title("☎ Contact Support")

st.write("**Email:** jagdevsinghdosanjh@gmail.com")
st.write("**WhatsApp:** +91-8146553307")

render_footer()

```

f. Utils

1. file_ops.py
No code here as yet
2. pdf_utils.py
No code here as yet
3. size_utils.py
No code here as yet

Home.py

```

import streamlit as st
from components.sidebar import render_sidebar
from components.footer import render_footer
from auth import (
    get_current_user,
    set_current_user,
    logout,
    supabase_sign_in,
    supabase_sign_up,
)
from backend.supabase_client import get_supabase_client

# -----
# MUST BE FIRST STREAMLIT COMMAND
# -----
st.set_page_config(page_title="CruncherX", layout="wide")
# st.session_state.clear()
# st.cache_resource.clear()
# st.cache_data.clear()

# -----
# INIT SUPABASE CLIENT
# -----
sb = get_supabase_client()

```

```

if sb is None:
    st.error("Supabase client failed to initialize. Check secrets.toml.")
    st.stop()

```

```

FREE_PLAN_ID = "a32d6731-8622-42df-b375-7309f478eab1"

```

```

# -----
# CREATE USER ROW IN public.profiles
# -----
def ensure_user_row(user):
    """Ensure a profile row exists for this user."""

    res = (
        sb.table("profiles")
        .select("id")
        .eq("id", user.id)
        .execute()
    )

    data = getattr(res, "data", res)

    if not data:
        sb.table("profiles").insert({
            "id": user.id,
            "email": user.email,
            "name": user.email.split("@")[0],
            "plan_id": FREE_PLAN_ID,
            "plan_expiry": None
        }).execute()

```

```

# -----
# HANDLE EMAIL CONFIRMATION REDIRECT
# -----
query_params = st.query_params

```

```

if "access_token" in query_params:
    token = query_params["access_token"]
    refresh = query_params.get("refresh_token", "")

```



```

# Set Supabase session
sb.auth.set_session(token, refresh)

auth_user = sb.auth.get_user()
user = getattr(auth_user, "user", None)

if user:
    ensure_user_row(user)

    set_current_user({
        "email": user.email,
        "id": user.id,
        "name": user.email.split("@")[0],
    })

    st.success("Email confirmed! You are now logged in.")
    st.rerun()

# -----
# LOGIN / SIGNUP VIEW
# -----
def login_view():
    st.title("🔒 CruncherX Login")

    tab_login, tab_signup = st.tabs(["Login", "Sign up"])

    # ----- LOGIN TAB -----
    with tab_login:
        email = st.text_input("Email", key="login_email")
        password = st.text_input("Password", type="password",
key="login_password")

        if st.button("Login"):
            if not email or not password:
                st.error("Please enter both email and password.")
            else:
                try:
                    res = supabase_sign_in(email, password)
                    user = getattr(res, "user", None)

```

```

        if user is None:
            st.error("Invalid credentials.")
        else:
            # Set session
            sb.auth.set_session(
                res.session.access_token,
                res.session.refresh_token
            )

            ensure_user_row(user)

            set_current_user({
                "email": user.email,
                "id": user.id,
                "name": user.email.split("@")[0],
            })
            st.rerun()

    except Exception as e:
        st.error(f"Login failed: {e}")

# ----- SIGNUP TAB -----
with tab_signup:
    new_email = st.text_input("Email", key="signup_email")
    new_password = st.text_input("Password", type="password",
key="signup_password")

    if st.button("Create account"):
        if not new_email or not new_password:
            st.error("Please enter email and password.")
        else:
            try:
                supabase_sign_up(new_email, new_password)
                st.success("Account created. Check your email to confirm.")
            except Exception as e:
                st.error(f"Sign up failed: {e}")

# -----
# DASHBOARD VIEW
# -----

```

```

def dashboard_view():
    render_sidebar()

    user = get_current_user()

    if user is None:
        st.error("Session expired. Please log in again.")
        st.rerun()

    st.title("🏠 CruncherX Dashboard")
    st.write(f"Welcome, **{user['name']}** ({user['email']})")
    st.write("Smaller PDFs. Bigger Productivity.")

    col1, col2 = st.columns([4, 1])
    with col2:
        if st.button("Logout"):
            logout()
            st.rerun()

    st.subheader("Choose a Mode:")
    st.page_link("pages/1_Cloud_Compressor.py", label="☁️ Cloud Compressor")
    st.page_link("pages/2_Local_Compressor.py", label="🏠 Local Bulldozer Mode")
    st.page_link("pages/3_OCR_Coming_Soon.py", label="🔍 OCR (Coming Soon)")
    st.page_link("pages/4_About_CruncherX.py", label="ℹ️ About CruncherX")
    st.page_link("pages/5_Contact_Support.py", label="📞 Contact Support")

    render_footer()

# -----
# ENTRY POINT
# -----
def main():
    user = get_current_user()

    if user is None:

```

```
login_view()
return
```

```
dashboard_view()
```

```
if __name__ == "__main__":
    main()
```

main_cloud.py

```
import streamlit as st
from backend.supabase_client import get_supabase_client
from engines.cruncher_cloud import compress_to_target

def main():
    st.title("☁ Cloud Compressor")

    supabase = get_supabase_client()

    # Auth context (you already have this)
    user = st.session_state.get("user")
    user_id = user["id"] if user else None
    org_id = user.get("org_id") if user else None

    uploaded_file = st.file_uploader("Upload PDF", type=["pdf"])

    if uploaded_file:
        input_path = f"/tmp/{uploaded_file.name}"
        with open(input_path, "wb") as f:
            f.write(uploaded_file.read())

        st.info("Compressing in cloud engine...")

        output_path, size_mb, status = compress_to_target(
            input_path=input_path,
            supabase=supabase,
            user_id=user_id,
            org_id=org_id
        )

        if status == "CloudError":
```

```
st.error("Cloud compression failed.")
return
```

```
st.success(f"Compression complete — {size_mb:.2f} MB")
```

```
with open(output_path, "rb") as f:
    st.download_button(
        label="Download Compressed PDF",
        data=f.read(),
        file_name=output_path,
        mime="application/pdf"
    )
```

```
if __name__ == "__main__":
    main()
```

.gitignore

```
# =====
# CruncherX — .gitignore
# Prevent .venv and unnecessary files
# =====

# --- Python ---
__pycache__/
*.pyc
*.pyo
*.pyd
*.pdb

# --- Virtual Environments ---
.venv/
local-venv/
venv/
env/
ENV/

# --- Streamlit ---
.streamlit/secrets.toml
.streamlit/config.toml
.streamlit/cache/
```

```
.streamlit/logs/

# --- Build / Output Files ---
dist/
build/
*.egg-info/
*.spec

# --- Logs ---
*.log
logs/
*.csv

# --- Temporary Files ---
*.tmp
*.temp
*.bak

# --- OS Junk ---
Thumbs.db
.DS_Store
desktop.ini

# --- IDE / Editor ---
.vscode/
.idea/
*.code-workspace

# --- PDF Processing Temp Files ---
temp/
tmp/
uploads/
output/
processed/
converted/

# --- Images / Cache ---
*.png
*.jpg
*.jpeg
```

*.gif

*.svg

(Optional) Keep branding assets:

!assets/logo.png

!assets/banner.png

--- Local Bulldozer Engine Dependencies ---

These should NOT be committed

qpdf/

poppler/

ghostscript/

bin/

--- Environment / Secrets ---

.env

.env.local

.env.production

.env.development

--- Git ---

.gitignore

local-requirements.txt

streamlit

pymupdf

pillow

pandas

bulldozer-req.txt

=====

CruncherX — Local Bulldozer Engine Requirements

Python 3.12.x (Local Only)

=====

--- Core ---

streamlit==1.32.0

python-dotenv==1.0.1

--- PDF Engines (Heavy, Local Only) ---

```
pikepdf==8.13.0
PyMuPDF==1.24.9
pypdf==4.2.0

# --- OCR (Local Only) ---
pytesseract==0.3.10

# --- Image Processing ---
Pillow==10.3.0
pdf2image==1.17.0

# --- Compression / Math ---
numpy==1.26.4

# --- Utilities ---
requests==2.31.0
urllib3==2.2.1
certifi==2024.2.2
charset-normalizer==3.3.2
idna==3.6

# --- Git Integration ---
GitPython==3.1.42
gitdb==4.0.11
smmap==5.0.1

# --- JSON & Schema ---
jsonschema==4.21.1
jsonschema-specifications==2023.12.1
referencing==0.34.0
rpds-py==0.18.0
MarkupSafe==2.1.5
Jinja2==3.1.3

# --- Time & Date ---
python-dateutil==2.9.0.post0
pytz==2024.1
tzdata==2024.1

# --- Visualization (Optional) ---
```



```
altair==5.2.0
plotly==5.18.0
pydeck==0.9.1

# --- Packaging ---
packaging==23.2
six==1.16.0
```

requirements.txt

```
# =====
# CruncherX — Optimized Requirements (Python 3.11)
# Cloud Engine + Local Bulldozer Engine
# =====

# --- Core App ---
streamlit==1.32.0
python-dotenv==1.0.1

# --- Supabase SDK (Stable for Python 3.11) ---
supabase==2.3.5
gotrue==2.4.2
postgrest==0.15.0
realtime==1.0.0
storage3==0.5.3
httpx==0.24.1

# --- PDF Handling (Cloud-Safe) ---
pypdf==4.2.0
pdf2image==1.17.0
Pillow==9.5.0

# --- Compression / Math ---
numpy==1.26.4

# --- Utilities ---
requests==2.31.0
urllib3==2.2.1
certifi==2024.2.2
charset-normalizer==3.3.2
```

idna==3.6

--- Streamlit Extensions ---

st-social-media-links==0.1.5

--- JSON & Schema ---

jsonschema==4.21.1

jsonschema-specifications==2023.12.1

referencing==0.34.0

rpds-py==0.18.0

MarkupSafe==2.1.5

Jinja2==3.1.3

--- Git Integration ---

GitPython==3.1.42

gitdb==4.0.11

smmmap==5.0.1

--- Time & Date ---

python-dateutil==2.9.0.post0

pytz==2024.1

tzdata==2024.1

--- Visualization ---

altair==5.2.0

plotly==5.18.0

pydeck==0.9.1

--- Packaging ---

packaging==23.2

six==1.16.0

setup_cruncherx.ps1

Write-Host "=== CruncherX Environment Reset ===" -

ForegroundColor Cyan

Step 1 — Remove old virtual environment

if (Test-Path ".venv") {

Write-Host "Removing old .venv..." -ForegroundColor Yellow

```
Remove-Item -Recurse -Force ".venv"  
}
```

```
# Step 2 — Create new virtual environment  
Write-Host "Creating new virtual environment..." -  
ForegroundColor Cyan  
python -m venv .venv
```

```
# Step 3 — Activate environment  
Write-Host "Activating environment..." -ForegroundColor Cyan  
.\venv\Scripts\activate
```

```
# Step 4 — Upgrade pip  
Write-Host "Upgrading pip..." -ForegroundColor Cyan  
python -m pip install --upgrade pip
```

```
# Step 5 — Install requirements  
Write-Host "Installing requirements.txt..." -ForegroundColor Cyan  
pip install -r requirements.txt
```

```
# Step 6 — Run CruncherX  
Write-Host "Launching CruncherX..." -ForegroundColor Green  
streamlit run app/Home.py
```

D. docs

E.

F.